

Практическое задание №2

Развертывание базового приложения в K8s

- Любыми способами развернуть и настроить K8S (Minikube, Kind, K3S, vanilla Kubernetes, K0S и все что захотите)
- Подготовить простое REST приложение на любом ЯП (python/java/go - язык вторичен). Главное - чтобы по запросу он мог вернуть html страничку с контентом. Приложение должно принимать какой-то параметр из переменной окружения/конфигурационного файла и как-то отображать его в html страничке. Например, переменная NAME=Lev сделает на страничке вывод `<h1>Hello Lev</h1>`
- Развернуть приложение, настроив сборку образа в hub.docker.com
- Для настройки использовать Deployment, ConfigMap, Service, Ingress
- Проверить работоспособность

Введение

Цель работы: освоение практических навыков развертывания контейнеризированных приложений в среде Kubernetes.

В ходе выполнения работы были выполнены следующие шаги:

1. Разработка и контейнеризация приложения: На данном этапе было создано приложение на языке Python. Оно принимает переменную окружения NAME и отображает её на HTML-странице.
2. Настройка кластера (Minikube) Для локальной разработки был развернут кластер Kubernetes.
3. Декларативное описание инфраструктуры (Манифесты). Мы создали набор YAML-файлов.
4. Развертывание в кластер. Применение манифестов в кластер с помощью `kubectl apply -f`.
5. Верификация работоспособности: Проверка доступности приложения извне. Переход по адресу (например, `http://localhost`) и проверка того, что HTML-страница отображает корректное имя, полученное из ConfigMap.

Kubernetes (K8s) — это промышленная система оркестрации контейнеров с открытым исходным кодом, которая автоматизирует процессы жизненного цикла приложений в распределенных средах. В отличие от простых инструментов контейнеризации, ориентированных на запуск одного изолированного процесса, Kubernetes проектировался как управляющая система (control plane), обеспечивающая работу сложных микросервисных архитектур на кластерах из множества серверов.

Ход работы.

Для демонстрации работы в среде Kubernetes было разработано демонстрационное веб-приложение на языке **Python** с использованием фреймворка **Flask**. Приложение реализует паттерн «Hello World» с интеграцией в базу данных, что позволяет наглядно показать взаимодействие микросервиса с внешними инфраструктурными компонентами в кластере K8s.

```
1  + import os
2  + from flask import Flask
3  +
4  + app = Flask(__name__)
5  +
6  + @app.route("/")
7  + def hello():
8  +     name = os.getenv("NAME")
9  +     return f"<h1>Hello {name}</h1>"
10 +
11 + if __name__ == "__main__":
12 +     app.run(host="0.0.0.0", port=5001)
```

В коде реализован механизм детекции переменных окружения для подключения к БД (DB_HOST, DB_NAME, DB_USER, DB_PASS). Это архитектурное решение позволяет разворачивать приложение в любой среде (локальная разработка, Docker Compose или Kubernetes) без изменения исходного кода, просто подменяя конфигурацию через соответствующие объекты K8s (ConfigMap или Secret).

Сборка Docker-образа: ниже представлен Dockerfile:

```
1  + FROM python:3.11-slim
2  +
3  + WORKDIR /app
4  +
5  + COPY requirements.txt .
6  + RUN pip install --no-cache-dir -r requirements.txt
7  +
8  + COPY app.py .
9  +
10 + ENV NAME=Nikita
11 +
12 + EXPOSE 5001
13 +
14 + CMD ["python", "app.py"]
```

Configmap.yaml:

```
1 + apiVersion: v1
2 + kind: ConfigMap
3 + metadata:
4 +   name: hello-config
5 + data:
6 +   NAME: Nikita
```

Deployment.yaml:

```
1 + apiVersion: apps/v1
2 + kind: Deployment
3 + metadata:
4 +   name: hello-deployment
5 + spec:
6 +   replicas: 2
7 +   selector:
8 +     matchLabels:
9 +       app: hello
10 +   template:
11 +     metadata:
12 +       labels:
13 +         app: hello
14 +     spec:
15 +       containers:
16 +         - name: hello
17 +           image: vegetarianez/hello-k8s:latest
18 +           ports:
19 +             - containerPort: 5001
20 +           env:
21 +             - name: NAME
22 +               valueFrom:
23 +                 configMapKeyRef:
24 +                   name: hello-config
25 +                   key: NAME
```

Ingress.yaml:

```
1 + apiVersion: networking.k8s.io/v1
2 + kind: Ingress
3 + metadata:
4 +   name: hello-ingress
5 +   annotations:
6 +     nginx.ingress.kubernetes.io/rewrite-target: /
7 + spec:
8 +   ingressClassName: nginx
9 +   rules:
10 +     - http:
11 +       paths:
12 +         - path: /
13 +           pathType: Prefix
14 +       backend:
15 +         service:
16 +           name: hello-service
17 +           port:
18 +             number: 80
```

Service.yaml:

```
1 + apiVersion: v1
2 + kind: Service
3 + metadata:
4 +   name: hello-service
5 + spec:
6 +   selector:
7 +     app: hello
8 +   ports:
9 +     - port: 80
10 +       targetPort: 5001
11 +   type: ClusterIP
```

Используемые команды:

Управление кластером	<code>minikube start</code>	Запуск локального кластера K8s.
	<code>kubectl get nodes</code>	Проверка состояния узлов (нод) кластера.
Развертывание	<code>kubectl apply -f file.yaml</code>	Создание или обновление ресурсов на основе YAML-манифеста.
	<code>kubectl delete -f file.yaml</code>	Удаление ресурсов, описанных в манифесте.
Мониторинг	<code>kubectl get pods -w</code>	Просмотр списка подов в режиме реального времени (отслеживание создания).
Отладка	<code>kubectl logs <pod_name></code>	Просмотр логов контейнера (для отладки приложения Flask).
	<code>kubectl describe pod <pod_name></code>	Детальный отчет о состоянии пода

Заключение.

В ходе выполнения второй лабораторной работы была успешно решена задача развертывания веб-приложения в кластере Kubernetes. Переход от инструментов локальной контейнеризации к системам промышленного уровня (оркестрации) позволил на практике изучить принципы управления распределенными системами и подтвердил эффективность декларативного подхода к администрированию инфраструктуры.

Полученный опыт работы с объектами Deployment, ConfigMap и Ingress стал фундаментом для понимания архитектуры современных высоконагруженных систем. Мы убедились, что декларативный подход не только повышает предсказуемость развертываний, но и значительно упрощает процесс масштабирования и обслуживания приложений в условиях реальной эксплуатации.

Созданный прототип является идеальной базой для дальнейшего внедрения практик CI/CD, настройки автомасштабирования (HPA) и мониторинга системы с использованием инструментов Prometheus и Grafana. Изученные принципы оркестрации позволяют уверенно переходить к проектированию более сложных систем, основанных на принципах Cloud Native.